STOCKMIND

Autonomous Multi-Agent Inventory Monitoring and Supply Chain Optimization System

Assignment 1.1: Project Ideation, Environment Modeling and PEAS Matrix Definition

Project field	Definition
Agent persona	StockMind Inventory Manager Agent
Domain	Retail warehouse inventory monitoring and supplier replenishment
Primary language	Python
Interface	Streamlit dashboard and terminal demonstrations
Agent interaction	Buyer / Inventory Agent and Supplier Agent
Operational scope	Educational simulation; simulated stock and supplier quotations; human approval before purchase

Design objective

Design a goal-directed agent that observes warehouse inventory, detects shortages, determines replenishment quantities, obtains supplier offers, evaluates alternatives and recommends a purchase while respecting budget and delivery constraints.

Project boundary

StockMind demonstrates autonomous agent behavior using a simulated warehouse and simulated supplier quotations. It does not connect to a real ERP, place real orders or process payments. A live weather REST API is an optional external knowledge source used as a simple delivery-risk signal.

1. Domain Definition and Problem Statement

Domain: Retail inventory management and supplier replenishment. A small warehouse stores multiple products. Stock levels change because of sales and other simulated environmental events. Each product has a reorder threshold and a target stock quantity. When stock reaches or falls below its reorder threshold, the agent should seek replenishment options.

Problem statement: Manual inventory checks can delay shortage detection and supplier coordination. StockMind automates observation, shortage detection, replenishment planning, supplier communication and offer evaluation. It aims to reduce stockout risk and procurement cost while accounting for delivery time, supplier reliability and product quality.

Agent Persona

Name: StockMind Inventory Manager Agent. Role: Monitor inventory and coordinate replenishment. Goal: Maintain stock availability while selecting a feasible supplier offer that provides the highest utility.

Restrictions: It cannot approve its own real-world purchase. Purchase execution remains a human-controlled step.

Main operational workflow

Step	Agent behavior
1. Perceive	Read current stock, reorder level, target stock and product metadata.
2. Reason	Apply condition-action rules to detect low stock, stockout or overstock.
3. Plan	Calculate the replenishment quantity and create an intention to request supplier quotations.
4. Communicate	Send a typed JSON request to the Supplier Agent over a TCP socket.
5. Coordinate	Exchange counteroffers and reach an agreement if reservation constraints overlap.
6. Optimize	Filter infeasible supplier offers and rank the remaining options using a weighted utility function.
7. Adapt	Update beliefs and replan when stock, supplier availability or weather risk changes.
8. Protect	Verify signed messages before accepting external instructions.
9. Recommend	Present a purchase recommendation and request human approval.

2. PEAS Profile

PEAS element	StockMind specification
Performance measure	Minimize stockouts; minimize purchase cost; keep orders within budget; meet quantity and delivery constraints; prefer reliable suppliers and acceptable product quality; maintain responsiveness and safe message handling.
Environment	Simulated retail warehouse; product inventory; sales/stock changes; reorder thresholds; supplier agents; supplier quotations; network channel; optional external weather forecast; human inventory manager.
Actuators	Create low-stock alerts; calculate reorder quantity; send supplier requests; send negotiation proposals; rank supplier offers; update BDI intentions; write event logs; display recommendation; request human approval.
Sensors	Inventory records; stock-change events; reorder-level configuration; supplier quote messages; supplier availability; message authentication result; weather API JSON response; user requirements and approval input.

Performance measure detail

The agent is evaluated by whether it detects threshold violations, calculates the correct reorder quantity, finds at least one feasible supplier offer when one exists, selects the highest-utility feasible offer, revises its plan after a relevant belief change, processes background updates without blocking the main interface, and rejects untrusted or modified messages.

3. Environment Characteristics Classification

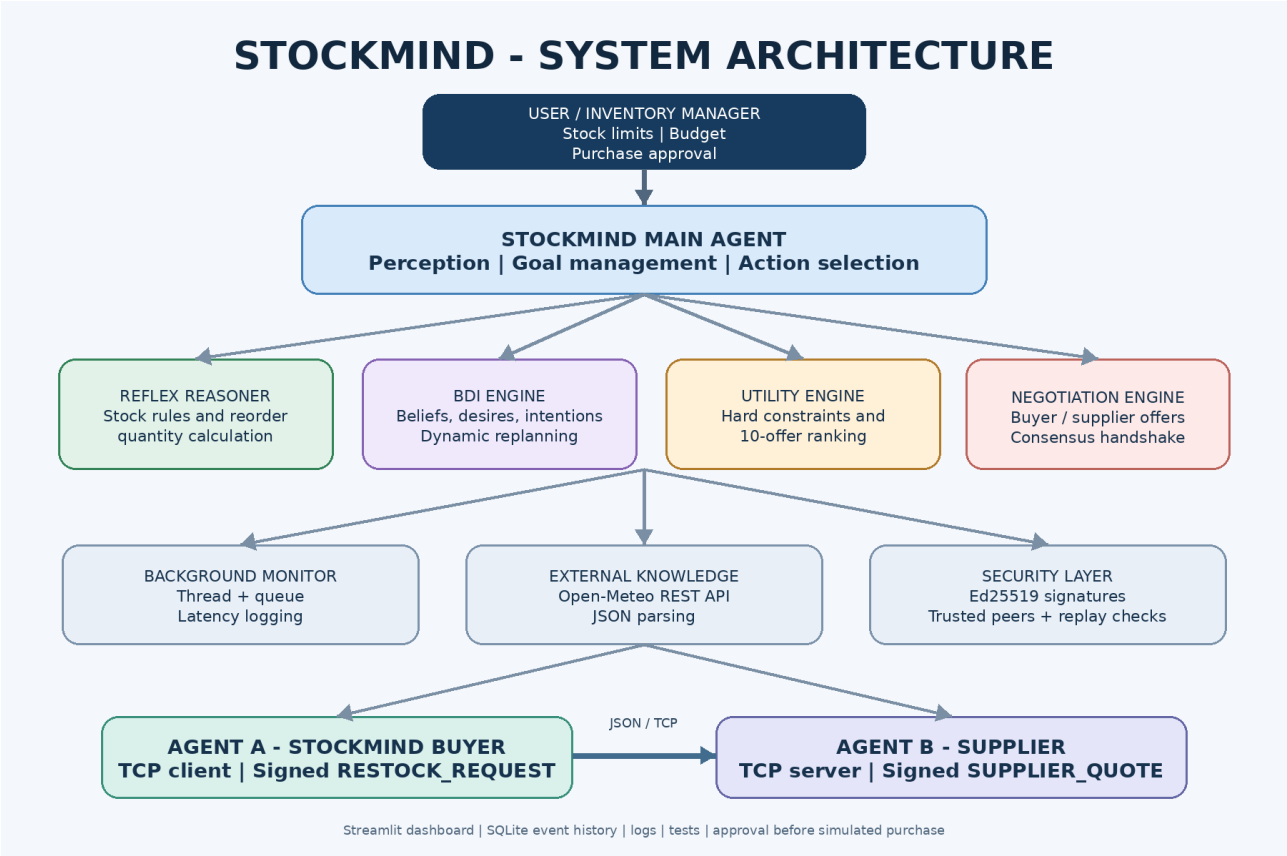
Criterion	Classification	Reason
Observability	Partially observable	The agent sees its warehouse records and received offers, but cannot inspect all supplier inventory, private supplier costs or future demand.
Determinism	Stochastic	Stock, demand, supplier availability and delivery conditions may change; the same action may not always produce the same future state.
Episode structure	Sequential	A replenishment decision affects later stock, orders, available budget and future decisions.
Dynamics	Dynamic	Inventory changes while the agent operates, and external conditions can change.
State representation	Mostly discrete	The prototype uses discrete products, stock counts, offers, messages and workflow states; numeric quantities and prices are represented as numbers.
Agent population	Multi-agent	A buyer/inventory agent communicates and negotiates with a supplier agent.
Knowledge	Partially known	The system knows its configured rules and observed records but not all future demand or supplier policies.

4. Autonomous Agent vs Standard Object-Oriented Program

StockMind is implemented using Python classes, so object-oriented programming is a useful implementation technique. The distinction is in the system behavior: a conventional OOP application typically executes methods in a prearranged control flow, whereas the agent architecture repeatedly senses its environment, maintains a state/goal model, evaluates options and selects an action based on current conditions.

Comparison	Standard OOP program	StockMind autonomous agent
Execution flow	A caller invokes methods according to a predefined workflow.	A recurring perceive-reason-act cycle chooses the next action from current observations and goals.
Input handling	Methods receive parameters or explicit events.	Sensors/percepts provide changing environment state.
Decision logic	Control flow is usually explicitly specified by the caller.	Rules, beliefs, intentions and utility jointly guide action selection.
State	Objects store attributes.	Belief state represents the agent’s current knowledge of the environment.
Goal	Completes a requested procedure.	Pursues persistent goals such as preventing stockouts and replenishing inventory.
Adaptation	A new path must be called or handled in the programmed flow.	The agent can re-evaluate beliefs and replan after an unexpected event.
Interaction	Calls functions or services when instructed.	Exchanges typed messages with another agent and participates in negotiation.
Autonomy	The application owner generally determines the operation sequence.	The agent selects an action within configured limits; purchase still requires human approval.

5. High-Level Architecture



The system is divided into perception/environment, reflex reasoning, BDI planning, utility optimization, negotiation, background monitoring, external knowledge, communication and security modules. The buyer agent and supplier agent exchange signed JSON messages over TCP. The Streamlit UI exposes the operational state and demonstration controls.

6. Agent Goals, Rules and Decision Constraints

Primary desires are to avoid stockouts, minimize replenishment cost and maintain reliable supply. The prototype's core rules are:

Condition	Condition-action rule	Action
Current stock = 0	ZERO_STOCK	Raise critical alert and create urgent replenishment request.
0 < stock <= reorder level	LOW_STOCK	Create replenishment request.
reorder level < stock <= target stock	STOCK_NORMAL	Continue monitoring.
stock > target stock	OVERSTOCK	Flag for excess-inventory review.

Replenishment quantity

Reorder quantity = max(0, Target stock - Current stock). Example: if Milk has 8 litres and the target stock is 100 litres, the requested replenishment quantity is 92 litres.

Supplier offer utility

The prototype computes a 0-100 score using the following weighted function:

U = 0.40 x C + 0.25 x D + 0.20 x R + 0.15 x Q

C is cost affordability, D is delivery convenience, R is supplier reliability and Q is product quality. Before ranking, the agent removes offers that cannot supply the required quantity, exceed the maximum unit budget or violate the maximum delivery time. The selected offer is the highest-utility offer among those that meet all hard constraints.

Safety and approval limits

Supplier messages are verified using Ed25519 digital signatures, trusted public keys, timestamps and nonces. Invalid messages are rejected. The prototype only recommends an order; a human must approve any purchase action. Demo keys and simulated offers are for coursework only.

7. Scope, Assumptions and Limitations

Item	Definition
Inventory data	Local JSON demonstration data; editable through the dashboard.
Supplier prices and offers	Simulated data included in the project; not live retailer quotes.
External API	Open-Meteo forecast API provides external weather data when internet access is available.
Weather interpretation	A simple configurable heuristic for delivery-risk awareness; not a courier delay prediction.
Network deployment	Local TCP demonstration on 127.0.0.1:5055; not a production deployment.
Security	Ed25519 signatures verify sender key and message integrity; they do not prove a host machine is uncompromised.
Purchasing	No real purchase, payment or inventory-system writeback occurs. Human approval is required.

8. Deliverable Reference

The complete project contains the executable Python modules, Streamlit dashboard, test suite, execution logs, socket client/server scripts, utility results for ten supplier offers, API integration, architecture image, presentation script, slide outline, and GitHub publishing instructions. See README.md and docs/Assignment_to_Deliverables_Mapping.md for the complete assignment-by-assignment evidence map.

End of Assignment 1.1 Design Specification